

Multi-relational graph attention networks for knowledge graph completion

Zhifei Li^{a,*}, Yue Zhao^a, Yan Zhang^{a,*}, Zhaoli Zhang^b

^a School of Computer Science and Information Engineering, Hubei University, Wuhan 430062, China

^b National Engineering Research Center for E-Learning, Central China Normal University, Wuhan 430079, China

ARTICLE INFO

Article history:

Received 8 January 2022

Received in revised form 8 May 2022

Accepted 11 June 2022

Available online 21 June 2022

Keywords:

Multi-relational learning

Knowledge graph completion

Graph neural network

Attention mechanism

ABSTRACT

Knowledge graphs are multi-relational data that contain massive entities and relations. As an effective graph representation technique based on deep learning, graph neural network has reported outstanding performance for modeling knowledge graphs in recent studies. However, previous graph neural network-based models have not fully considered the heterogeneity of knowledge graphs. Furthermore, the attention mechanism has demonstrated its great potential in many areas. In this paper, a novel heterogeneous graph neural network framework based on a hierarchical attention mechanism is proposed, including entity-level, relation-level, and self-level attentions. Thus, the proposed model can selectively aggregate informative features and weights them adequately. Then the learned embeddings of entities and relations can be utilized for the downstream tasks. Extensive experimental results on various heterogeneous graph tasks demonstrate the superior performance of the proposed model compared to several state-of-the-art methods.

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

Knowledge graphs (KGs) are multi-relational data in which nodes are represented as entities and edges are denoted as relations between entities. An edge can be organized as the triplet form of (*subject entity*, *relation*, *object entity*) to indicate the concrete fact. Numerous of real-world KGs [1–3] have been created and applied in a variety of applications, including recommendation systems [4–8] question answering [9–12], and dialogue generation [13–15]. In both academia and industry communities, considerable attention has been paid to KGs.

However, the KGs are generally confronted with problems such as partialness, completeness, and newly-added information. Predicting the missing links is a fundamental problem for KG completion [16,17]. Extensive studies have been focused on the statistical relational learning (SRL) [18]. The basic idea of SRL is to obtain the low-dimensional continuous embeddings of relations and entities (a.k.a., KG embedding [19–21]). The learned embeddings can preserve the inherent structure of KGs and be used to promote subsequent link prediction tasks.

Recently, graph neural network (GNN) [22] has been proposed and shown superior performance for graph-structured data [23,24]. Some models [25–27] apply the convolutional operation

to graph data and propose the graph convolutional networks. Although GNN demonstrates impressive ability for graph representation learning, most of the previous studies have focused on learning embeddings of nodes in non-relational graphs. For a multi-relational graph, we argue that it should learn not only the embeddings of nodes, but also the embeddings of relations. Hence, the design of a powerful GNN framework for KGs needs to be considered.

In contrast to the homogeneous graphs, the KGs are generally recognized as the heterogeneous information network (HIN) [28] which contains multiple categories of entities and relations. The heterogeneity of KGs is usually reflected by the relation-path [29], which demonstrates complex semantic features that involve different triplets. There are one-step or multiple-step relation-paths between entities. As displayed in Fig. 1, there are four categories of entities and three categories of relations. The entity *oilfield* involves various entity categories, including the *worker* and the *factory* through different relation types. Each relation-path can convey a specific semantic information that involves the triplet. Taking the relation-path *work_for* as an example, the central node *oilfield* can only aggregate the semantic information of the *worker*. It is not reasonable to fuse all neighbor nodes for each entity directly. Thus, reserving multiple feature information simultaneously and handling such a complex multi-relational graph is an urgent issue.

The attention mechanism has become a recent research trend in the GNN. It has shown the effectiveness in different GNN-based architectures and been employed to various tasks [30,

* Corresponding author.

E-mail addresses: zhifei1993@hubu.edu.cn (Z. Li), zhangyan@hubu.edu.cn (Y. Zhang).

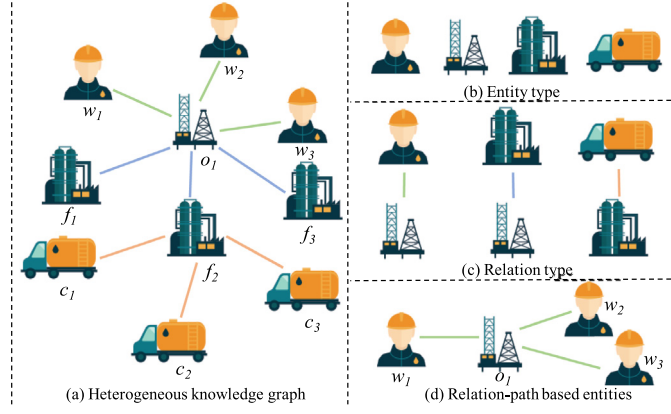


Fig. 1. An illustrative example of heterogeneous KGs. (a) The KGs consists of multiple types of entities connected by various types of relations. (b) Four types of entities (i.e., worker, oilfield, factory, and car). (c) Three types of relations (i.e., worker-work, worker-for-factory). (d) The Oilfield o_1 and its relation-path based neighbors (i.e., w_1 , w_2 , and w_3).

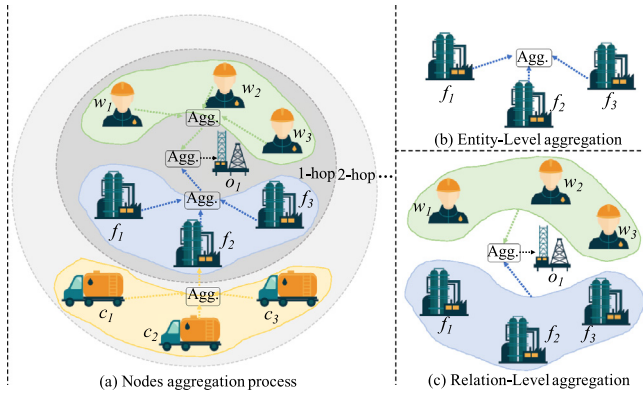


Fig. 2. An illustrative architecture of MRGAT. (a) The nodes aggregation process in heterogeneous knowledge graph mainly contains two steps. (b) The first step intends to merge all relation-path based neighbor features. (c) The second step aims to fuse different categories of semantic features about the various relation-paths.

31]. However, the attention mechanism has not been fully considered for the multi-relational graph. As a matter of fact, the relation-path [32] generally reflects the heterogeneity of KGs. Different relation-paths can aggregate diverse semantic information. Considering each relation-path equally is impractical, where aggregated semantic features in some crucial relation-paths will be weakened. Furthermore, given a relation-path, each entity has many relation-path based neighbors. It is required to select some informative neighbors. Hence, the attention mechanism can be utilized to select the important relation-paths and neighbors, and allocate appropriate weights to them.

In this paper, the novel multi-relational graph attention networks (MRGAT) are presented to effectively and efficiently learn the entity and relation embeddings. In particular, as shown in Fig. 2, the MRGAT can be regarded as the heterogeneous GNN which fuses neighbor node features through the hierarchical aggregation. First, the entity-level part intends to merge all relation-path based neighbor entities. Second, the relation-level aggregation aims to fuse different categories of semantic features about the various relation-paths. Moreover, the attention mechanism is integrated into both entity-level and relation-level aggregation to select the useful relation-paths and neighbors. Based on the preceding steps, the optimal combination of relation-path and neighbor features can be aggregated through a hierarchical manner. Then the learned embeddings can better model the complex structure and reflect the rich semantic information in the KGs.

The main contributions of our work can be summarized as follows:

- We design a heterogeneous GNN-based framework for KGs, which learns the entity and relation embeddings simultaneously. Our work makes it possible to directly apply GNN to the multi-relational graph, and then facilitates the following link prediction task.
- We propose the novel multi-relational graph attention networks (MRGAT). The MRGAT fuses neighbor features through the entity-level aggregation and relation-level aggregation based on the attention mechanism, which can selectively aggregate informative features.
- We conduct comprehensive experiments on real-world KGs to show the performance of the MRGAT. The results show that the proposed model has a superior performance compared to the state-of-the-art models on the task of link prediction, graph classification, and node classification. Moreover, the MRGAT demonstrates its interpretability by analyzing the attention mechanism.

The rest of this paper is organized as follows. Section 2 gives the review and analysis of related work. Section 3 provides the background. The details of the proposed model are presented in Section 4. The experimental results on graph datasets are reported in Section 5. The summary and future work are concluded in Section 6.

2. Related work

In this section, we first introduce the related studies on the GNN, and then review the advances on multi-relational learning in the KGs. At last, the latest studies of the GNN for multi-relational graph are presented.

2.1. Graph neural network

Graph neural network (GNN) [22] generalizes the deep neural network for non-Euclidean data. Kipf & Welling [26] first propose the graph convolutional networks (GCN) to simplify the graph convolutions. In follow-up works, several extensions have been formulated. FastGCN [33], as an efficient variant of the GCN, which is strengthened with importance sampling for inductive node classification. GraphSAGE [27] fuses neighbor feature information based on node sampling and disparate aggregation function. SGC [34] is proposed to reduce redundant complexity. It

observes that the nonlinearity transformation is burdensome and unnecessary, and replaces it as a single linear transformation.

The attention mechanism (e.g., self-attention [35] or soft-attention [36]), as a significant learning technique, has been successfully utilized in many graph-based tasks [5,6]. Based on the attention mechanism, GAT [30] learns the attention values between neighbor nodes for node classification. HAN [31] proposes the semi-supervised heterogeneous GNN based on the hierarchical attention. However, most of aforementioned models are established for the homogeneous graph. They cannot straight employ in the heterogeneous multi-relational graph.

2.2. Multi-relational learning

Multi-relational learning in KGs aims to embed the entities and relations into continuous vector spaces. Started from a effective translational distance-based model called TransE [37], a range of variants have been proposed, such as TransH [38], TransR [29], MGTransR [39], and TransD [40]. Moreover, TorusE [41] preserves the translational property on a lie group. RotatE [42] and MRotatE [43] proposes the novel embedding model which defines a rotation operation in the complex vector space. The typical semantic matching-based model DistMult [44] embeds the entities and relations through a bilinear operation. ComplEx [45] extends DistMult to model asymmetric relations in the complex vector space. PARL [46] develops a novel text-enhanced relation prediction architecture to vary the sparsity and reliability of KGs. Some surveys [17–19] give detailed comparisons of above models.

The recent ConvE [47] proposes the convolutional network-based framework which adopt a multi-layer of 2D convolution over embeddings, and obtains the state-of-the-art link prediction results. ConvTransE [48] removes the reshape operation in ConvE, which can keep the translational property as TransE. ConvKB [49] constrains the convolution to be in the same embedding dimensions, which replaces the 2D convolution with the 1D convolution. Nevertheless, few previous models incorporate connectivity structure and have structure enforcement in the embedding space. We can aggregate neighbor information for each entity in KGs, then used for predicting missing links.

2.3. GNN for multi-relational graph

A well-known variant of GNN for multi-relational graph is R-GCN [50] which demonstrates its effectiveness in the link prediction and entity classification. To incorporate neighbor feature information, W-GCN [48] leverages graph structure and attributes of graph nodes. Furthermore, W-GCN can adaptively learn the weights for each subgraph when combining the embeddings for a node. KBGAT [51] learns GAT-based embeddings that specifically cater to link prediction on the KGs. Given a central entity, KBGAT captures both relation and entity information in a multihop neighborhood. CompGCN [52] proposes a more generic structure which simultaneously embeds entities and relations in KGs. CAFE [53] presents an approach to complete KGs based on evaluating candidate triples using a set of neighborhood-based features. In this paper, our proposed model designs a heterogeneous GNN-based framework based on the attention mechanism. We compare the detailed differences and experiment results against aforementioned method in Section 4 and Section 5, respectively.

Table 1
Notations and explanations.

Notations	Explanations	Notations	Explanations
$\mathcal{G}$	Multi-relational graph	$\mathcal{V}$	Entities & Nodes
$\mathcal{E}$	Edges & Triplets	$\mathcal{R}$	Relations
$\mathbf{E}$	Entity embeddings	$\mathbf{R}$	Relation embeddings
s/o	Subject/object entity	r	Relation
$\mathbf{h}_i$	Node feature	$\mathbf{h}_r$	Relation feature
$\mathcal{N}_i$	Local neighbors	α	Node attention
β	Relation-path attention	γ	Self-attention
$\mathbf{W}$	Weight matrix	σ	Activation function
$\phi(\cdot)$	Aggregation function	$\psi(\cdot)$	Score function

The **bold** letter denotes the vector, e.g. $\mathbf{r}$ corresponding to r .

3. Preliminaries

The multi-relational graph is the typical kind of heterogeneous information network which consists of multiple categories of entities and relations. In this section, we present relevant concepts utilized in this paper. In Table 1, the main notations utilized throughout this paper are summarized.

Definition 1 (Multi-Relational Graph). The multi-relational graph $\mathcal{G}$ contains a node set $\mathcal{V}$, a relation set $\mathcal{R}$, and an edge set $\mathcal{E}$. The edge set $\mathcal{E}$ in the multi-relational graph can be represented as $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{R} \times \mathcal{V}$. The relation set $\mathcal{R}$ denotes the link types between nodes, where $|\mathcal{R}| > 2$. As shown in Fig. 1(a) and (b), there are four types of entities and three types of relations. The entity *oilfield* involves various entity types including the *worker* and the *factory* through different relation types.

Definition 2 (Relation-path). An relation-path is defined as a path from the subject node to the object node in the form of $s \xrightarrow{r_1} e_1 \xrightarrow{r_2} e_2 \cdots e_{n-1} \xrightarrow{r_n} o$. And $r_1 \circ r_2 \cdots \circ r_n$ describes a composite relation between s and o , where $\circ$ refers the composition operator on relations. As presented in Fig. 1(c), three nodes of the *worker* can be connected via single meta-path *work_for*. Furthermore, disparate meta-paths always reveal disparate semantics.

Definition 3 (Feature Propagation). Feature propagation is what distinguishes a GCN from an Multi-Layer Perceptron (MLP). Let $\mathbf{h}_i^{(l)}$ represent the node feature of the l th layer, the formula $\mathbf{h}_i^{(l+1)} = f(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)})$ denotes the forward propagation, where f is the nonlinear transformation and $j \in \mathcal{N}_i$ denotes the local neighborhood.

Definition 4 (Link Prediction). The aim of link prediction for KGs is to forecast another reasonable entity to compose a correct triplet. For instance, given the subject entity s and the relation r , predicting the object entity o . To achieve this goal, a general methodology is to define a score function $\psi(s, r, o) \in \mathbb{R}$ for the triplet. In general, the aim of the optimization is to score correct triplets higher than incorrect triplets.

4. The proposed MRGAT

In this section, we formally present a detailed description of the proposed MRGAT. MRGAT consists of three parts: (1) entity-level aggregation; (2) relation-level aggregation; (3) self-level aggregation. The overall architecture is shown in Fig. 3. The entity-level aggregation is proposed to first aggregate each relation-path based entity feature. In the relation-level aggregation part, various types of semantic information are aggregated through the relation-paths related to the entity. Finally, we incorporate the self-loops into the feature propagation in the self-level aggregation.

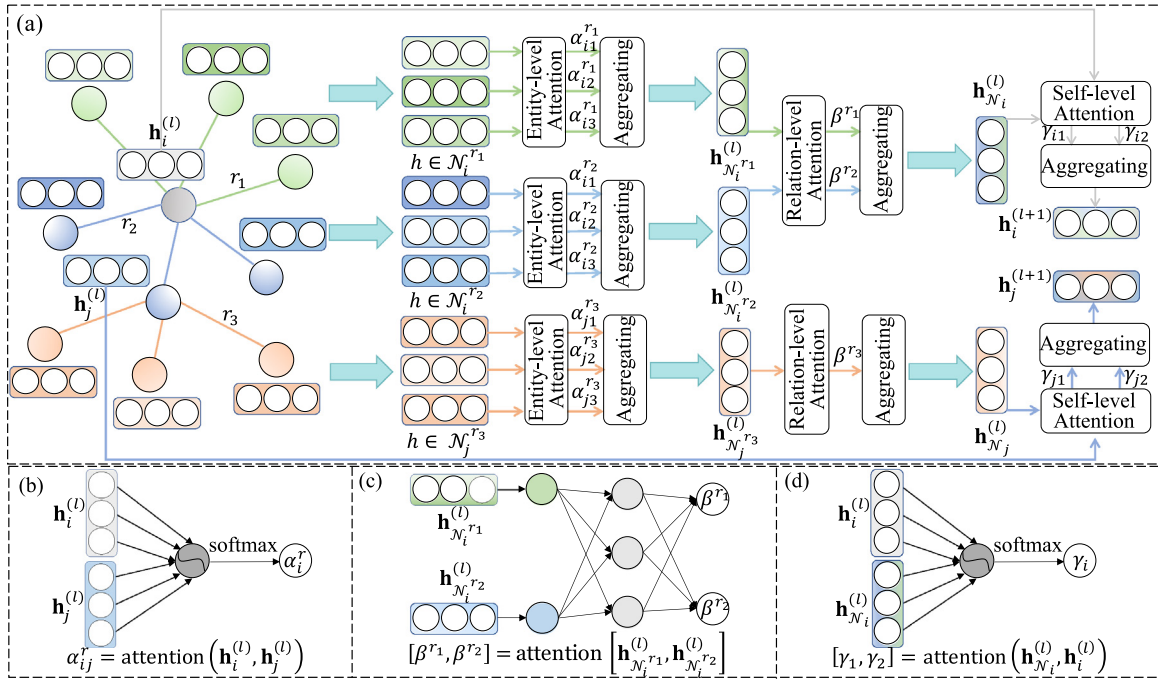


Fig. 3. (a) Illustration of the proposed MRGAT framework. For each entity, MRGAT first aggregates each relation-path based neighbors features through the entity-level attention. Second, the different semantic information of various relation-paths are fused through the relation-level attention. Finally, the aggregated neighbors features and the entity itself feature are aggregated through the self-level attention. (b) The entity-level attention part; (c) The relation-level attention part; (d) The self-level attention part.

4.1. Entity-level aggregation

Due to the heterogeneity of multi-relational graph (e.g. KGs), different categories of entities may be in specific embedding spaces. Aggregating all neighbor features for each entity directly is not suitable. Thus, we intend to fuse relation-path based node features firstly. The obtaining features in entity-level aggregation will be semantic-specific and can capture one type of semantic information. Taking the input entity and relation features $\mathbf{h}_i^{(l)}, \mathbf{h}_r^{(l)} \in \mathbb{R}^d$ as the input, the entity-level feature propagation is proposed to first aggregate each relation-path based entity features as follows:

$$\mathbf{h}_{N_i^r}^{(l)} = \phi(\mathbf{h}_j^{(l)}), \forall j \in N_i^r \quad (1)$$

where N_i^r denotes a set of relation-path based entities. The symbols $\mathbf{h}_j^{(l)}$ and $\mathbf{h}_{N_i^r}^{(l)}$ are the j -th entity and aggregated in the l -layer, respectively.

After that, the self-attention mechanism is utilized to learn the weight among neighbor entities. The process can be shown as follows:

$$\alpha_{ij}^r = \text{attention}(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}), \quad (2)$$

where $\mathbf{h}_i^{(l)}$ is the feature of central entity. The importance α_{ij}^r of neighbor j -th entity can be learned through the *attention* function. In this paper, we follow in GAT [30] and the attention function is denoted as follows:

$$e_{ij}^r = \mathbf{a}^\top (\mathbf{W}_n \mathbf{h}_i^{(l)}, \mathbf{W}_n \mathbf{h}_j^{(l)}), \quad (3)$$

where $\mathbf{W}_n$ is transformation weight matrix, $\mathbf{a}$ is the attention vector. We normalize the weight coefficient α_{ij}^r after calculating the importance between neighbor entities via softmax function:

$$\alpha_{ij}^r = \text{softmax}_j(e_{ij}^r) = \frac{\exp(e_{ij}^r)}{\sum_{k \in N_i^r} \exp(e_{ik}^r)}, \quad (4)$$

in which

$$\alpha_{ij}^r = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}_n \mathbf{h}_i^{(l)} \parallel \mathbf{W}_n \mathbf{h}_j^{(l)}]))}{\sum_{k \in N_i^r} \exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}_n \mathbf{h}_i^{(l)} \parallel \mathbf{W}_n \mathbf{h}_k^{(l)}]))}, \quad (5)$$

where LeakyReLU is the activation function and $\parallel$ represents the concatenate operation. Then, the relation-path r based features $\mathbf{h}_{N_i^r}^{(l)}$ of entity i can be aggregated by the neighbor features with the corresponding coefficients as follows:

$$\mathbf{h}_{N_i^r}^{(l)} = \frac{1}{|N_i^r|} \sum_{j \in N_i^r} \alpha_{ij}^r \mathbf{h}_j^{(l)}. \quad (6)$$

Since the KGs show the property of scale free, which have quite high variance. To tackle this issue, the entity-level attention is extended to multi-head attention to obtain more stable results. Specifically, the entity-level attention is repeated for K times and the learned features are summed as follows:

$$\mathbf{h}_{N_i^r}^{(l)} = \frac{1}{|N_i^r|} \frac{1}{K} \sum_{k=1}^K \sum_{j \in N_i^r} \alpha_{ij}^{r(k)} \mathbf{h}_j^{(l)}. \quad (7)$$

4.2. Relation-level aggregation

Diverse categories of semantic information are fused by the relation-paths in the relation-level aggregation stage. we can observe that entities reflect various types of semantic information due to the heterogeneity of KGs. It can only get information from one aspect for each semantic-specific aggregated feature. To merge more complete semantic information, numerous features reflected by diverse relation-paths must be aggregated. Furthermore, it will recede the semantic features fused by main relation-paths if regarding each relation-path uniformly. To solve these problems, we proposed a novel attention mechanism based on the relation-path. It aggregates diverse categories

Algorithm 1: Multi-Relational Graph Attention Networks algorithm.

Input: The multi-relational graph $\mathcal{G} = (\mathcal{V}, \mathcal{R}, \mathcal{E})$,
The dimension of feature d ,
The number of network layer L ,
The number of attention head K ,
Output: The entity embedding matrices $\mathbf{E}$,
The relation embedding matrices $\mathbf{R}$.

- 1 Initialize node feature $\mathbf{h}_i^0 \in \mathbb{R}^d, \forall i \in \mathcal{V}$;
- 2 Initialize relation feature $\mathbf{h}_r^0 \in \mathbb{R}^d, \forall r \in \mathcal{R}$;
- 3 **for** $l = 0$ to L **do**
- 4 **for** $r \in \mathcal{R}$ **do**
- 5 **for** $j \in \mathcal{V}$ **do**
- 6 $\alpha_{ij}^r \leftarrow \text{attention}(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}), \forall j \in \mathcal{N}_i^r$;
 // Calculate the node-attention value.
- 7 $\mathbf{h}_{\mathcal{N}_i^r}^{(l)} \leftarrow \frac{1}{|\mathcal{N}_i^r|} \frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}_i^r} \alpha_{ij}^{r(k)} \mathbf{h}_j^{(l)}$; // Fuse each
 relation-specific node features.
- 8 **end**
- 9 $\beta^r \leftarrow \text{attention}(\mathbf{h}_{\mathcal{N}_i^r}^{(l)}), \forall r \in \mathcal{R}$; // Calculate the
 relation-attention value.
- 10 $\mathbf{h}_{\mathcal{N}_i}^{(l)} \leftarrow \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} (\beta^r \mathbf{h}_{\mathcal{N}_i^r}^{(l)} + \mathbf{h}_r^{(l)})$; // Fuse all
 relation-specific node features.
- 11 **end**
- 12 $[\gamma_1, \gamma_2] \leftarrow \text{attention}(\mathbf{h}_{\mathcal{N}_i}^{(l)}, \mathbf{h}_i^{(l)})$; // Calculate the
 self-attention value.
- 13 $\mathbf{h}_i^{(l+1)} \leftarrow \text{ReLU}((\gamma_1 \mathbf{h}_{\mathcal{N}_i}^{(l)} + \gamma_2 \mathbf{h}_i^{(l)}) \mathbf{W}^{(l)})$; // Update node
 features.
- 14 $\mathbf{h}_r^{(l+1)} \leftarrow \text{ReLU}(\mathbf{h}_r^{(l)} \mathbf{W}^{(l)})$; // Update relation features.
- 15 **end**
- 16 $\mathbf{E} \leftarrow \text{concat} \{ \mathbf{h}_i^{(L)}, \forall i \in \mathcal{V} \}$; // Return entity embeddings.
- 17 $\mathbf{R} \leftarrow \text{concat} \{ \mathbf{h}_r^{(L)}, \forall r \in \mathcal{R} \}$. // Return relation
 embeddings.

semantic information by learning the importance of different relation-paths.

Unlike most previous models which only embed entities, MR-GAT also embeds a d -dimensional embedding $\mathbf{h}_r \in \mathbb{R}^d$ of relation r . Specifically, the relation-level aggregation can be represented as follows:

$$\mathbf{h}_{\mathcal{N}_i}^{(l)} = \phi(\mathbf{h}_r^{(l)}, \mathbf{h}_{\mathcal{N}_i^r}^{(l)}), \forall r \in \mathcal{R}. \quad (8)$$

To obtain the weights of each relation-path, $|\mathcal{R}|$ groups of entity-level aggregated features are utilized as the input. The process is as follows:

$$[\beta^{r_1}, \beta^{r_2}, \dots, \beta^{r_{|\mathcal{R}|}}] = \text{attention} \left[\mathbf{h}_{\mathcal{N}_i^{r_1}}^{(l)}, \mathbf{h}_{\mathcal{N}_i^{r_2}}^{(l)}, \dots, \mathbf{h}_{\mathcal{N}_i^{r_{|\mathcal{R}|}}}^{(l)} \right], \quad (9)$$

where *attention* denotes the attention function. It can capture the relation-based importance β^r then utilized to selectively aggregate informative features behind a heterogeneous KGs.

In this paper, we follow the squeeze-and-excitation operation [54] and squeeze the global feature information $\mathbf{h}_{\mathcal{N}_i^r}^{(l)}$ into a relation-specific descriptor z_r . This is achieved by using global

average pooling as follows:

$$z_r = \text{squeeze}(\mathbf{h}_{\mathcal{N}_i^r}^{(l)}) \\ = \frac{1}{d} \sum_1^d \mathbf{h}_{\mathcal{N}_i^r}^{(l)}, \quad (10)$$

where d is the dimension of $\mathbf{h}_{\mathcal{N}_i^r}^{(l)}$. After that, we can obtain each statistic about relation-path aggregated feature as follow:

$$\mathbf{z} = [z_{r_1}, z_{r_2}, \dots, z_{r_{|\mathcal{R}|}}]. \quad (11)$$

To utilize aggregated information $\mathbf{z}$ in the *squeeze* operation, the second operation *excitation* is adopted to capture dependencies between each relation-path aggregated feature $\mathbf{h}_{\mathcal{N}_i^r}^{(l)}$. To achieve this goal, a simple gating mechanism is utilized to capture their dependencies

$$[\beta^{r_1}, \beta^{r_2}, \dots, \beta^{r_{|\mathcal{R}|}}] = \text{excitation}(\mathbf{z}) \\ = \sigma(\mathbf{W}_a \text{ReLU}(\mathbf{W}_b \mathbf{z})), \quad (12)$$

where σ refers to the sigmoid activation function. $\mathbf{W}_a$ is a dimensionality-increasing transformation matrix and $\mathbf{W}_b$ is a dimensionality-increasing transformation matrix. This two fully-connected (FC) layers are utilized to capture the dependencies.

Finally, all neighbor aggregated feature $\mathbf{h}_{\mathcal{N}_i}^{(l)}$ around node i can be obtained as follows:

$$\mathbf{h}_{\mathcal{N}_i}^{(l)} = \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} (\beta^r \mathbf{h}_{\mathcal{N}_i^r}^{(l)} + \mathbf{h}_r^{(l)}), \quad (13)$$

where $\mathbf{h}_r^{(l)}$ denotes the relation features. We utilize the translation distance operation used in TransE [37] by incorporating relation features into the feature propagation.

4.3. Self-level aggregation

In (13), the input vectors of all neighbor features are summed up but not the entity feature $\mathbf{h}_i^{(l)}$ itself. Hence, we incorporate the self-loops into the feature propagation as follows:

$$\mathbf{h}_i^{(l+1)} = \phi(\mathbf{h}_{\mathcal{N}_i}^{(l)}, \mathbf{h}_i^{(l)}), \quad (14)$$

where $\mathbf{h}_i^{(l+1)}$ is the feature of entity i in the $(l+1)$ -layer. To learn the importance γ_1, γ_2 of aggregated features $\mathbf{h}_{\mathcal{N}_i}^{(l)}$ and entity features itself $\mathbf{h}_i^{(l)}$, the operation is denoted as follows:

$$[\gamma_1, \gamma_2] = \text{attention}(\mathbf{h}_{\mathcal{N}_i}^{(l)}, \mathbf{h}_i^{(l)}). \quad (15)$$

It first transforms the features $\mathbf{h}_{\mathcal{N}_i}^{(l)}$ or $\mathbf{h}_i^{(l)}$ utilizing a nonlinear MLP transformation. The significance of features is then measured through the attention vector. The activation function determines the final *gamma*. The procedure is depicted as follows:

$$w_k = \mathbf{q}^\top \tanh(\mathbf{W}_s \mathbf{h}_k^{(l)} + \mathbf{b}), \quad \forall k \in [\mathcal{N}_i, i] \quad (16)$$

where the transformation weight matrix is represented by $\mathbf{W}_s$, the bias vector is represented by $\mathbf{b}$, and the attention vector is represented by $\mathbf{q}$. The parameters mentioned above, in particular, are independent of entity or relation features. Obviously, the higher w_k gets, the more significant features are.

Finally, the weight of features $\mathbf{h}_{\mathcal{N}_i}^{(l)}$ and $\mathbf{h}_i^{(l)}$ can be obtained by normalizing the above importance of all w_k using softmax function

$$\gamma = \frac{\exp(w_k)}{\exp(w_{\mathcal{N}_i}) + \exp(w_i)}, \quad (17)$$

Table 2
Comparison with other graph convolutional models.

Models	Feature propagation	Node feature	Relation feature	Attention mechanism	Graph heterogeneity
R-GCN [50]	$\mathbf{h}_i^{(l+1)} = \sigma \left(\sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_i^r} \frac{1}{ \mathcal{N}_i^r } \mathbf{h}_j^{(l)} \mathbf{W}_r^{(l)} + \mathbf{h}_i^{(l)} \mathbf{W}^{(l)} \right)$	✓	✗	✗	✓
KBGAT [51]	$\mathbf{h}_i^{(l+1)} = \sigma \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}_i^k} \alpha_{ij}^k \mathbf{h}_j^{(l)} \mathbf{W}^{(l)} \right)$	✓	✗	✓	✗
W-GCN [48]	$\mathbf{h}_i^{(l+1)} = \sigma \left(\sum_{j \in \mathcal{N}_i} \alpha_r^{(l)} \mathbf{h}_j^{(l)} \mathbf{W}^{(l)} + \mathbf{h}_i^{(l)} \mathbf{W}^{(l)} \right)$	✓	✗	✓	✗
CompGCN [52]	$\mathbf{h}_i^{(l+1)} = \sigma \left(\sum_{j \in \mathcal{N}_i} \phi \left(\mathbf{h}_j^{(l)}, \mathbf{h}_r^{(l)} \right) \mathbf{W}_r^{(l)} \right)$	✓	✓	✗	✗
MRGAT	$\mathbf{h}_i^{(l+1)} = \sigma \left(\phi \left(\sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_i^r} \phi \left(\mathbf{h}_{j \in \mathcal{N}_i^r}^{(l)}, \mathbf{h}_r^{(l)} \right), \mathbf{h}_i^{(l)} \right) \mathbf{W}^{(l)} \right)$	✓	✓	✓	✓

then we can update the entity and relation feature through the non-linearity transformation as

$$\mathbf{h}_i^{(l+1)} = \text{ReLU} \left(\frac{1}{2} \left(\gamma_1 \mathbf{h}_{\mathcal{N}_i}^{(l)} + \gamma_2 \mathbf{h}_i^{(l)} \right) \mathbf{W}^{(l)} \right), \quad (18)$$

and

$$\mathbf{h}_r^{(l+1)} = \text{ReLU} \left(\mathbf{h}_r^{(l)} \mathbf{W}_r^{(l)} \right), \quad (19)$$

where the entity-specific and relation-specific connection coefficient matrices are represented by $\mathbf{W}^{(l)}$ and $\mathbf{W}_r^{(l)}$, respectively. Finally, the embedding matrices of entities and relations can be obtained by

$$\begin{cases} \mathbf{E} = \text{concat} \left\{ \mathbf{h}_i^{(L)}, \forall i \in \mathcal{V} \right\} \\ \mathbf{R} = \text{concat} \left\{ \mathbf{h}_r^{(L)}, \forall r \in \mathcal{R} \right\} \end{cases}. \quad (20)$$

In summary, MRGAT not only aggregates entity features from various semantic aspects, but also allocates sufficient weights to them. It is capable of capturing various forms of semantic information and selectively fusing informative features. The overall process is shown in Algorithm 1. The difference of MRGAT with existing GNN-based models for multi-relational graph are show in Table 2.

4.4. Link prediction and model training

For link prediction in KGs, taken an triplet (s, r, o) as input, the embedding of entity and relation should be first identified in $\mathbf{E}$ and $\mathbf{R}$, respectively. It can be obtained as

$$\begin{cases} \mathbf{s} = \mathbf{x}_s^\top \mathbf{E} \\ \mathbf{r} = \mathbf{x}_r^\top \mathbf{R} \\ \mathbf{o} = \mathbf{x}_o^\top \mathbf{E} \end{cases}, \quad (21)$$

where $\mathbf{x}_s^\top$, $\mathbf{x}_r^\top$, and $\mathbf{x}_o^\top$ denote the high-dimensional one-hot index vectors of s , r , and o .

In this paper, we opt the TransE [37], DistMult [44], and ConvE [47] models as the decoder for link prediction tasks. Specifically, the convolution decoder ConvE achieves best performance and the prediction probability of a triplet (s, r, o) is defined as

$$\begin{aligned} \hat{y}(s, r, o) &= \sigma(\psi(\mathbf{s}, \mathbf{r}, \mathbf{o})) \\ &= \sigma(\text{ReLU}(\text{vec}(\text{ReLU}(\bar{\mathbf{s}}; \bar{\mathbf{r}}) * \Omega)) \mathbf{W}) \mathbf{o}) \end{aligned} \quad (22)$$

where $\bar{\cdot}$ denotes the 2D reshaping, σ denotes sigmoid activation function, $[\cdot]$ denotes the concatenation operation, and $*$ denotes convolution operation. The symbol Ω represents the filters and $\mathbf{W}$ is linear transformation matrix.

In KGs, if the triplet (s, r, o) is valid, we expect its probability $\hat{y}(s, r, o)$ equal to 1; otherwise, the probability is equal to 0. Then the loss function for MRGAT can be defined as follows:

$$\min \mathcal{L} = -\frac{1}{N} \sum_{\substack{(s, r, o) \in \mathcal{E} \\ (s, r, o') \in \mathcal{E}'}} \left(y \log \hat{y}(s, r, o) + (1 - y) \log (1 - \hat{y}(s, r, o')) \right), \quad (23)$$

Algorithm 2: The training process of link prediction task.

Input: The train triplet set $\mathcal{E}$,
The embedding matrices $\mathbf{E}$ and $\mathbf{R}$,
The batch size b ,
The score function $\psi(\cdot)$,
Output: The triplet prediction probability $\hat{y}(\cdot)$.

```

1 for  $\mathcal{E}_{batch} \leftarrow \text{sample}(\mathcal{E}, b)$  do
2    $S_{batch} \leftarrow \emptyset$ ; // Initialize train triplets set.
3   for  $(s, r, o) \in \mathcal{E}_{batch}$  do
4     for  $o' \in \mathcal{V}$  do
5        $\mathcal{E}'_{batch} \leftarrow (s, r, o'), \wedge (s, r, o') \notin \mathcal{E}$ ;
6       // Generate the negative triplets set.
7     end
8      $S_{batch} \leftarrow S_{batch} \cup \{(s, r, o), (s, r, o')\}$ ; // Generate the train triplets set.
9      $\begin{cases} \mathbf{s} \leftarrow (s, \mathbf{E}) \\ \mathbf{r} \leftarrow (r, \mathbf{R}) \\ \mathbf{o} \leftarrow (o, \mathbf{E}) \\ \mathbf{o}' \leftarrow (o', \mathbf{E}) \end{cases}$ ; // Look up the embeddings.
10     $\hat{y}(s, r, o) \leftarrow \psi(\mathbf{s}, \mathbf{r}, \mathbf{o})$ ; // Return the positive triplet prediction probability.
11     $\hat{y}(s, r, o') \leftarrow \psi(\mathbf{s}, \mathbf{r}, \mathbf{o}')$ ; // Return the negative triplet prediction probability.
12  end
13 Back propagation and update parameters by (23).
```

in which

$$y = \begin{cases} 1 & \text{if } (s, r, o) \in \mathcal{E} \\ 0 & \text{if } (s, r, o') \in \mathcal{E}' \end{cases}, \quad (24)$$

where the set of incorrect triplets are represented as $\mathcal{T}'$ which are generated by corrupting the correct triplet set $\mathcal{T}$.

The training processes of link prediction tasks are shown in Algorithm 2. The MRGAT is regularized by using the dropout technique [55] in the training process. Batch normalization [56] is adopted to regularize the rate of convergence after each layer. The label smoothing [57] is utilized to lessen overfitting and improve generalization. As an efficient tool for gradient-based optimization, the Adam optimizer [58] is utilized to optimize the loss function.

5. Experiments

In this section, the effectiveness of MRGAT is evaluated compared to many state-of-the-art models for various tasks. Extensive experiments and analyses are carried out to discuss the following research questions:

Table 3
Statistics of the datasets.

Tasks	Datasets	# Graphs	# Entities	# Relations	# Edges _(train)	# Classes	Mean degree
Link prediction	FB15k-237	1	14,541	237	310,116 _(272,115)	–	18.71
	WN18RR	1	40,943	11	93,003 _(86,835)	–	2.12
	NELL-995	1	75,492	200	154,213 _(149,678)	–	1.98
Graph classification	MUTAG _(Graph)	188	17.9 (Avg.)	4	39.6 (Avg.)	2	–
	PTC-MR	344	25.5 (Avg.)	4	29.5 (Avg.)	2	–
Node classification	MUTAG _(Node)	1	23,644	23	74,227	2	3.14
	AIFB	1	8,285	45	29,043	4	3.51

- **RQ1.** How does MRGAT perform vs. the state-of-the-art models for various tasks, such as link prediction, graph classification, and node classification?
- **RQ2.** How do different components, e.g., GCN encoders or link prediction decoder, affect the model performance?
- **RQ3.** How do various attention mechanisms, e.g., entity-level and relation-level attention, affect in the framework?
- **RQ4.** How do various hyper-parameters, e.g., GCN network layer or attention head, impact the model performance?

The relevant settings of the proposed model are shortly summarized before introducing the experiment results.

5.1. General settings

5.1.1. Datasets

Extensive experiments are carried out on the benchmark datasets listed below. **Link prediction:** FB15k-237 [59], WN18RR [47], and NELL-995 [60]. **Graph classification:** MUTAG_(Graph) [61] and PTC-MR [23]. **Node classification:** MUTAG_(Node) [61] and AIFB [62]. The detailed information and statistics of these datasets are shown in Table 3.

5.1.2. Evaluation metrics

To assess prediction accuracy, three general evaluation metrics are adopted. It consists of mean rank (MR), mean reciprocal rank (MRR), and Hits@k (for $k = 1, 3$, and 10).

Three general evaluation metrics are utilized to measure prediction accuracy. It includes mean rank (MR), mean reciprocal rank (MRR), and Hits@k (where $k = 1, 3$, and 10). They are represented as follows:

$$\begin{cases} \text{MR:} & \frac{1}{2|\mathcal{E}_{\text{test}}|} \sum_{i \in \mathcal{E}_{\text{test}}} (\text{rank}_i^s + \text{rank}_i^o) \\ \text{MRR:} & \frac{1}{2|\mathcal{E}_{\text{test}}|} \sum_{i \in \mathcal{E}_{\text{test}}} \left(\frac{1}{\text{rank}_i^s} + \frac{1}{\text{rank}_i^o} \right) \\ \text{Hits@k:} & \frac{1}{2|\mathcal{E}_{\text{test}}|} \sum_{i \in \mathcal{E}_{\text{test}}} I[\text{rank}_i^s \leq k] + I[\text{rank}_i^o \leq k] \end{cases}, \quad (25)$$

where average ranks and average inverse ranks are represented as MR and MRR, respectively. The percentage of ranks lower than or equal to k is represented as Hits@k. It means better performance when the lower MR and higher MRR or Hits@k is.

5.1.3. Comparison models

To investigate the effectiveness of our proposed model, from the field of link prediction, we compare our model with various state-of-the-art baselines, including translational distance-based models (TransE [37], TorusE [41], and RotatE [42]), semantic matching-based models (DistMult [44] and ComplEx [45]), neural networks-based models (ConvE [47], ConvKB [49], and InteractE [63]), and GCN-based models (R-GCN [50], W-GCN [48], and CompGCN [52]). Moreover, to verify the effectiveness of proposed attention mechanisms, we also test three variants of MRGAT, including MRGAT (w/o n-a.), MRGAT (w/o r-a.), and MRGAT (w/o s-a.), which remove the entity-level attention, relation-level attention and self-level attention, respectively.

5.1.4. Implementation details

The hyper-parameters of MRGAT were selected via grid search during training as follows: entity and relation embedding dimension [50, 100, 200, 300], layer depth [1, 2, 3], filter number [36, 48, 72], filter size [2×3 , 2×5 , 2×7], label smoothing [0.1, 0.2, 0.3], batch size [64, 128, 256], learning rate [0.0001, 0.001, 0.005], and dropout rate sampled from 0.1 to 0.5. The best hyper-parameter values are chosen based on the MRR on the validation set: entity and relation embedding dimension of 200, layer depth of 1. For link prediction task, the parameters of batch size, filter number, and label smoothing are set as 128, 48, and 0.1 respectively. Specifically, for FB15k-237 and WN18RR datasets, the parameters of filter size, dropout rate, and learning rate are set as 2×7 , 0.4, and 0.0001 respectively. For NELL-995 dataset, the parameters of filter size, dropout rate, and learning rate are set as 2×5 , 0.3, and 0.001 respectively. We implemented our models and replicated the baselines DistMult, ComplEx, and ConvE via software library PyTorch [69]. For above baselines, we set the embedding dimension as 200, batch size as 128, filter size as 3×3 , and filter number as 64. The hyper-parameters and experimental results of other baselines are obtained from the original paper. And we conduct the experiments on a PC server equipped with Nvidia GTX 1080 Ti GPU.

5.2. Performance of MRGAT

5.2.1. Link prediction

In this part, the effectiveness of MRGAT and baseline models are evaluated. The conclusion of MRR and Hits@k on FB15k-237, WN18RR and NELL-995 datasets are presented in Table 4. We take the score results of baseline models from the previous papers [42,48,52]. The highest score is in **bold**. The findings can be outlined as follows:

In sum, the proposed MRGAT outperforms existing link prediction models on FB15k-237 dataset. And MRGAT shows better performance in 3 out of 4 metrics on WN18RR and NELL-995 datasets. The results demonstrates the effectiveness of proposed model on link prediction task. Moreover, comparing to three variants of MRGAT, MRGAT achieves better performance in all metrics on three datasets. It indicates that different components of the attention mechanism can enhance the performance of the model on the link prediction task. We also note that the baseline Complex and RotatE learn the embeddings in complex space. We intend to improve the link prediction performance by adopting a complex variant of our proposed model.

5.2.2. Graph & node classification

In this section, the graph and node classification tasks on four datasets are evaluated. Specifically, graph classification refers to classifying graph structure data made up of nodes and edges. And node classification aims to predict non-existing node properties based on other node properties. Table 5 gives the experimental results. The highest score is in **bold**. The findings can be outlined as follows:

Overall, on graph classification task, the proposed MRGAT achieves better performance compared to all the baseline models

Table 4
Link prediction performance on FB15k-237, WN18RR, and NELL-995 datasets.

Models		FB15k-237				WN18RR				NELL-995			
		MRR	Hits			MRR	Hits			MRR	Hits		
			@1	@3	@10		@1	@3	@10		@1	@3	@10
Baselines	TransE [37]	0.257	0.174	0.284	0.420	0.182	0.027	0.295	0.444	0.456	0.514	0.678	0.751
	DistMult [44]	0.241	0.155	0.263	0.419	0.430	0.390	0.440	0.490	0.522	0.610	0.704	0.795
	ComplEx [45]	0.247	0.158	0.275	0.428	0.440	0.410	0.460	0.510	0.652	0.614	0.784	0.815
	R-GCN [50]	0.249	0.151	0.264	0.417	0.123	0.080	0.137	0.207	0.120	0.082	0.126	0.188
	ConvE [47]	0.325	0.237	0.356	0.501	0.430	0.400	0.440	0.520	0.747	0.672	0.808	0.864
	ConvKB [49]	0.243	0.155	0.371	0.421	0.249	0.057	0.417	0.524	0.430	0.370	0.470	0.545
	TorusE [41]	0.316	0.217	0.335	0.484	0.452	0.422	0.464	0.512	0.568	0.602	0.689	0.756
	RotatE [42]	0.338	0.241	0.375	0.533	0.476	0.428	0.492	0.571	0.754	0.670	0.812	0.854
	W-GCN [48]	0.352	0.261	0.385	0.536	0.466	0.427	0.478	0.535	0.744	0.668	0.826	0.859
	InteractE [63]	0.354	0.263	0.386	0.535	0.463	0.430	0.483	0.528	0.751	0.681	0.816	0.872
CompGCN [52]	0.355	0.264	0.390	0.535	0.479	0.443	0.494	0.546	0.753	0.679	0.825	0.876	
Ablation	MRGAT (w/o n-a.)	0.348	0.261	0.386	0.532	0.475	0.437	0.486	0.535	0.751	0.665	0.817	0.870
	MRGAT (w/o r-a.)	0.346	0.258	0.379	0.530	0.468	0.431	0.479	0.531	0.746	0.660	0.811	0.864
	MRGAT (w/o s-a.)	0.352	0.263	0.387	0.536	0.478	0.445	0.489	0.540	0.752	0.669	0.820	0.876
	MRGAT	0.355	0.266	0.392	0.539	0.481	0.449	0.495	0.544	0.761	0.678	0.829	0.881

Table 5
Performance comparison on graph classification and node classification.

Graph classification			Node classification		
Models	MUTAG _(Graph)	PTC-MR	Models	MUTAG _(Node)	AIFB
PachySAN [64]	92.6 ± 4.2	60.0 ± 4.8	Feat [65]	77.9	55.6
DGCNN [66]	85.8	58.6	WL [67]	80.9	80.6
GIN [23]	89.4 ± 5.6	64.6 ± 7.0	RDF2Vec [61]	67.2	88.9
R-GCN [50]	82.3 ± 9.2	67.8 ± 13.2	R-GCN [50]	73.2	95.8
SynGCN [68]	79.3 ± 10.3	69.4 ± 11.5	SynGCN [68]	74.8 ± 5.5	95.7 ± 1.0
W-GCN [48]	78.9 ± 12.0	67.3 ± 12.0	W-GCN [48]	77.9 ± 3.2	96.1 ± 0.6
CompGCN [52]	89.0 ± 11.1	71.6 ± 12.0	CompGCN [52]	85.3 ± 1.2	96.3 ± 0.5
MRGAT	89.2 ± 9.4	72.6 ± 10.1	MRGAT	86.8 ± 2.1	95.7 ± 0.7

Table 6
Performance on link prediction task evaluated on FB15k-237 dataset.

Score function (=S) → Models ↓		TransE			DistMult			ConvE		
		MR	MRR	Hits@10	MR	MRR	Hits@10	MR	MRR	Hits@10
Baselines	S	357	0.294	0.465	354	0.241	0.419	244	0.325	0.501
	S + R-GCN	325	0.281	0.443	230	0.324	0.499	197	0.342	0.524
	S + W-GCN	1520	0.267	0.444	229	0.324	0.504	201	0.344	0.525
	S + CompGCN	214	0.336	0.518	227	0.335	0.514	197	0.355	0.535
Ablation	S + MRGAT (w/o n-a.)	238	0.337	0.516	221	0.329	0.514	198	0.347	0.530
	S + MRGAT (w/o r-a.)	244	0.331	0.509	227	0.326	0.512	210	0.342	0.525
	S + MRGAT (w/o s-a.)	233	0.340	0.518	217	0.331	0.516	191	0.350	0.534
S + MRGAT		231	0.343	0.525	212	0.334	0.522	186	0.355	0.539

Table 7
Results on link prediction by relation category on FB15k-237 dataset.

Models → Relation categories ↓		ConvE		W-GCN + ConvE		CompGCN + ConvE		MRGAT + ConvE	
		MRR	Hits@10	MRR	Hits@10	MRR	Hits@10	MRR	Hits@10
Subject entity prediction	1-to-1	0.193	0.385	0.422	0.547	0.457	0.604	0.468	0.612
	1-to-N	0.068	0.116	0.093	0.187	0.112	0.190	0.126	0.215
	N-to-1	0.438	0.638	0.454	0.647	0.471	0.656	0.486	0.678
	N-to-N	0.246	0.436	0.261	0.459	0.275	0.474	0.287	0.492
Object entity prediction	1-to-1	0.177	0.391	0.406	0.531	0.453	0.589	0.465	0.601
	1-to-N	0.756	0.867	0.771	0.875	0.779	0.885	0.793	0.894
	N-to-1	0.049	0.090	0.068	0.139	0.076	0.151	0.101	0.169
	N-to-N	0.369	0.587	0.385	0.607	0.395	0.616	0.409	0.628

on PTC-MR dataset. And on node classification task, we find that MRGAT also obtains a comparable performance. From the results, we can conclude that MRGAT also performs well on graph & node classification tasks. Furthermore, it can find that MRGAT and CompGCN outperform other baselines significantly. This demonstrates the effectiveness of incorporating the relation feature in GCN-based feature propagation.

5.3. Influence of different component

To investigate how different component contributes to the performance of link prediction task, we evaluate the performance of different GCN encoders (R-GCN, W-GCN, CompGCN, and MRGAT) with different link prediction decoders (TransE, DistMult, and ConvE). In the results, the GCN encoder is utilized to generate

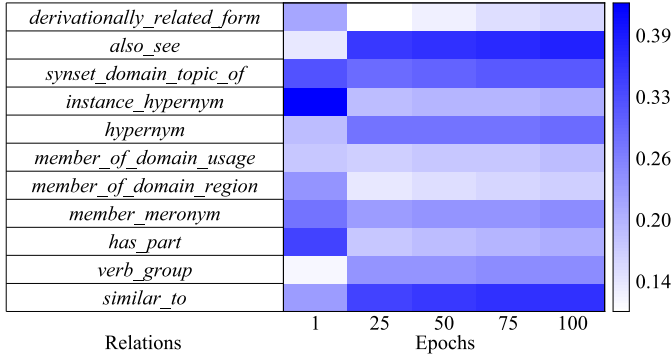


Fig. 4. Relation-path attention values varies with the increase of epoch number on the WN18RR dataset.

the entity embeddings (and relation embeddings in CompGCN and MRGAT). The link prediction decoder is used for the score function.

As shown in Table 6, it can observe that using the various GCN components as encoder gives a substantial improvement with most score functions. And MRGAT obtains better performance in 7 out of 9 metrics on three datasets. On average, MRGAT achieves around 12.9%, 24.5% and 7.6% relative improvement in Hits@10 with TransE, DistMult, and ConvE as the score function respectively. The results can be attributed to the reason that MRGAT selectively aggregates and learns the features of entities and relations via an attention-based GCN encoder.

The results for different relation categories on FB15k-237 dataset are summarized in Table 7. Following TransE [37], the relations in KGs can be divided into four categories: 1-1, 1-N, N-1, and N-N. We can find that our model MRGAT encoder with ConvE decoder outperforms all the baselines. These results demonstrates the usefulness of MRGAT when modeling both simple and complex relations in KGs.

5.4. Effect of attention mechanism

To assess the effect of hierarchical attention mechanism, we conduct several experiments on WN18RR and FB15k-237 datasets. In the training process, the attention values of different relation-paths can be learned through a specific task. Fig. 4 shows that all relation-path attention values varies with the increase of epoch number on the WN18RR dataset. We can observe that the attention values tends to be stable as epoch changes except that epoch one is random.

On FB15k-237 dataset, different relation-paths and the learned values are shown in Fig. 5. Here we take the entity *Los Angeles Lakers* as an illustrative example. The relation-path based neighbors of entity *Los Angeles Lakers* and their learned attention values are calculated. From Fig. 5, we can observe that *Los Angeles Lakers* connects to *Shooting guard* and *Power forward* through the relation-path *position*. From the results, it can see that the relation-path *position* gets the lower attention which indicates the relation-path based nodes plays unimportant role in learning its features.

Based on the above analysis, we can conclude that entity-level and relation-level attention mechanism can reveal the characteristic in relation-paths and neighbor entities. The proposed model MRGAT can selectively fuse informative features and weights them adequately.

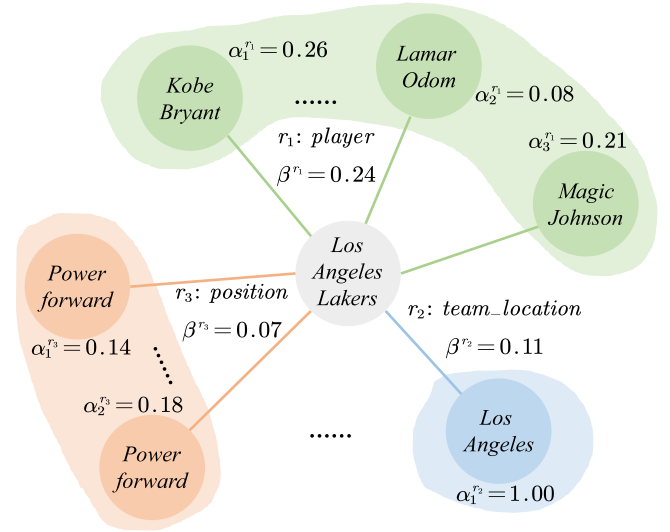


Fig. 5. Different relation-paths, neighbor nodes, and attention value on the FB15k-237 dataset.

5.5. Parameter sensitivity

5.5.1. Depth of network layer L

To investigate the influence of network layers, the efficiency of MRGAT and its variants with varying layer depth is report in Fig. 6. From the results on FB15k-237, WN18RR, and NELL-995 datasets, it can observe that the proposed model MRGAT and its variants obtain the better performance when the depth is equal to 1. The experimental results lead to be worse as the number of network layer increases. The key explanation for this fact is that as the layer of graph neural networks increases, the aggregated features will become overly smooth.

5.5.2. Number of attention head K

To investigate the effect of multi-head attention, we report the results of MRGAT and its two variants with varying number of attention head. We utilize the attention head $K = \{1, 2, 4, 6, 8\}$ for Hits@10 on three datasets and the results are shown in Fig. 7. When we set the number of attention head at 1, it can be regarded that the multi-head attention is removed. Based on the results, it can observe that the performance of MRGAT and its two variants will generally improve with the number of attention head increasing. And it achieves the best performance when the attention head $K = 6$. However, we can find that the change of attention head influences the results of proposed models slightly. From the results, it can conclude that the multi-head attention is an effective technique which makes the training process more stable.

5.6. Complexity analysis

The model complexity of MRGAT is $\mathcal{O}((|\mathcal{E}|d^2 + |\mathcal{R}|d)L)$, where d is the embedding dimension. $|\mathcal{E}|$ and $|\mathcal{R}|$ denote the total number of entities and relations, respectively. And L is the feature propagation of GCN layers. We compare the running time of MRGAT with W-GCN and CompGCN on FB15K-237. The results are as follows: R-GCN: 136s/epoch; CompGCN: 257s/epoch; MRGAT: 288s/epoch. The results indicate that MRGAT has a considerable performance improvement at an acceptable increase in computational requirements.

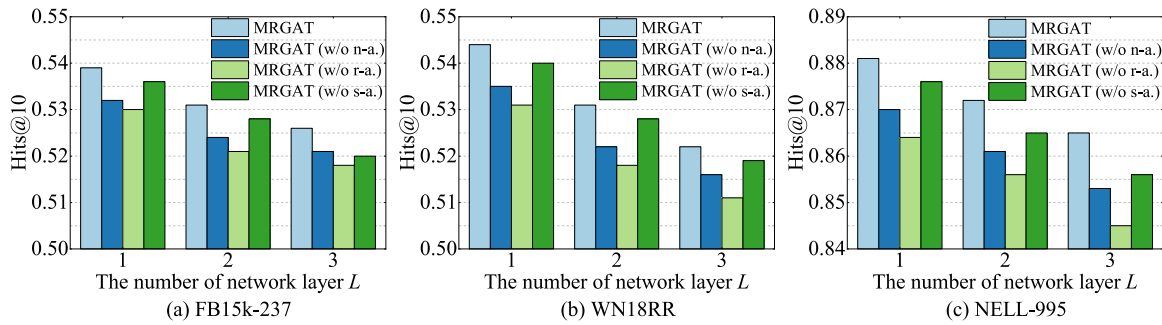


Fig. 6. Parameter sensitivity of network layer L on (a) FB15k-237, (b) WN18RR, and (c) NELL-995 datasets.

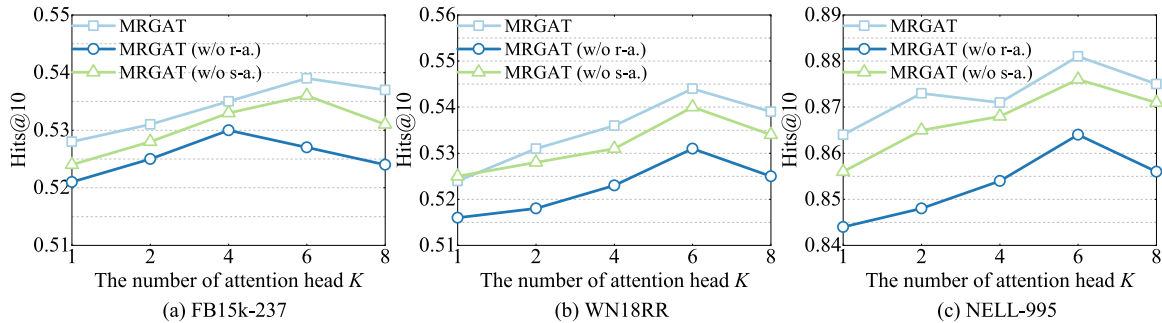


Fig. 7. Parameter sensitivity of attention head K on (a) FB15k-237, (b) WN18RR, and (c) NELL-995 datasets.

5.7. Defect analysis

The defect analysis in this paper mainly consists of three parts. Firstly, the proposed attention mechanism may inherit unnecessary complexity and redundant computation. We can reduce this excess complexity by removing nonlinearities and weight matrices in future work. Secondly, sampling useful incorrect training examples is a crucial task. We intend to assess if the performance of MRGAT can be further improved or not, by considering sampling useful incorrect triplets. Thirdly, each entity in a KG may also contain other background knowledge, extending MRGAT to exploit logical rules together with texts, images, etc can be considered.

6. Conclusion

In this paper, we propose the multi-relational graph neural network framework, called MRGAT, for various graph-based tasks. To capture complex semantics in heterogeneous KGs, MRGAT mostly relies on the attention mechanism. Specifically, the proposed model adopts entity-level attention, relation-level attention, and self-level attention to selectively aggregate informative features and weights them adequately. MRGAT incorporates both entity and relation features in the feature propagation. Thus, the structural information and the feature information of heterogeneous KGs can be utilized adequately. Experimental results include link prediction, graph classification, and node classification demonstrate the effectiveness of the proposed model. Moreover, the effect of different components and parameters are investigated in the experiments.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors sincerely thank anonymous reviewers for their constructive comments, which helped improve this paper. This work was supported in part by the National Natural Science Foundation of China under Grant 61977021, 62101179, 62077020 and Major Project of Hubei Province under Grant 2019ACA144 and School Project of Hubei University under Grant 202111903000001, 202011903000002 and China University Collaborative Innovation Fund under Grant 2020ITA05050.

References

- [1] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, J. Taylor, Freebase: a collaboratively created graph database for structuring human knowledge, in: Proc. ACM SIGMOD Int. Conf. Manage. Data, 2008, pp. 1247–1250.
- [2] G.A. Miller, WordNet: a lexical database for English, Commun. ACM 38 (11) (1995) 39–41.
- [3] A. Carlson, J. Betteridge, B. Kisiel, B. Settles, E.R. Hruschka, T.M. Mitchell, Toward an architecture for never-ending language learning, in: Proc. 24th AAAI Conf. Artif. Intell., 2010, pp. 1306–1313.
- [4] H. Wang, F. Zhang, J. Wang, M. Zhao, W. Li, X. Xie, M. Guo, Exploring high-order user preference on the knowledge graph for recommender systems, ACM Trans. Inf. Syst. 37 (3) (2019) 1–26.
- [5] X. Wang, X. He, Y. Cao, M. Liu, T.-S. Chua, KGAT: Knowledge graph attention network for recommendation, in: Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, 2019, pp. 950–958.
- [6] H. Wang, F. Zhang, M. Zhang, J. Leskovec, M. Zhao, W. Li, Z. Wang, Knowledge-aware graph neural networks with label smoothness regularization for recommender systems, in: Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, 2019, pp. 968–977.
- [7] H. Wang, F. Zhang, J. Wang, M. Zhao, W. Li, X. Xie, M. Guo, RippleNet: Propagating user preferences on the knowledge graph for recommender systems, in: Proc. 27th ACM Int. Conf. Inf. Knowl. Manage., 2018, pp. 417–426.
- [8] H. Wang, F. Zhang, X. Xie, M. Guo, DKN: Deep knowledge-aware network for news recommendation, in: Proc. 2018 World Wide Web Conf., 2018, pp. 1835–1844.
- [9] S. Hu, L. Zou, J.X. Yu, H. Wang, D. Zhao, Answering natural language questions by subgraph matching over knowledge graphs, IEEE Trans. Knowl. Data Eng. 30 (5) (2018) 824–837.

- [10] X. Huang, J. Zhang, D. Li, P. Li, Knowledge graph embedding based question answering, in: *Proc. 12th ACM Int. Conf. Web Search Data Mining*, 2019, pp. 105–113.
- [11] Y. Hao, H. Liu, S. He, K. Liu, J. Zhao, Pattern-revising enhanced simple question answering over knowledge bases, in: *Proc. 27th Int. Conf. Comput. Linguistics*, 2018, pp. 3272–3282.
- [12] M. Dubey, D. Banerjee, D. Chaudhuri, J. Lehmann, EARL: Joint entity and relation linking for question answering over knowledge graphs, in: *Proc. 17th Int. Semantic Web Conf.*, 2018, pp. 108–126.
- [13] H. He, A. Balakrishnan, M. Eric, P. Liang, Learning symmetric collaborative dialogue agents with dynamic knowledge graph embeddings, in: *Proc. 55th Annu. Meeting Assoc. Comput. Linguistics*, 2017, pp. 1766–1776.
- [14] A. Madotto, C. Wu, P. Fung, Mem2Seq: Effectively incorporating knowledge bases into end-to-end task-oriented dialog systems, in: *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics*, 2018, pp. 1468–1478.
- [15] H. Zhou, T. Young, M. Huang, H. Zhao, J. Xu, X. Zhu, Commonsense knowledge aware conversation generation with graph attention, in: *Proc. 27th Int. Joint Conf. Artif. Intell.*, 2018, pp. 4623–4629.
- [16] A re-evaluation of knowledge graph completion methods, in: *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 5516–5522.
- [17] X. Chen, S. Jia, Y. Xiang, A review: Knowledge reasoning over knowledge graph, *Expert Syst. Appl.* 141 (2020).
- [18] A review of relational machine learning for knowledge graphs, *Proc. IEEE* 104 (1) (2016) 11–33.
- [19] Q. Wang, Z. Mao, B. Wang, L. Guo, Knowledge graph embedding: A survey of approaches and applications, *IEEE Trans. Knowl. Data Eng.* 29 (12) (2017) 2724–2743.
- [20] A. Rossi, D. Barbosa, D. Firmani, A. Matinata, P. Merialdo, Knowledge graph embedding for link prediction: A comparative analysis, *ACM Trans. Knowl. Discov. Data* 15 (2) (2021) 1–49.
- [21] S. Ji, S. Pan, E. Cambria, P. Marttinen, P.S. Yu, A survey on knowledge graphs: Representation, acquisition, and applications, *IEEE Trans. Neural Netw. Learn. Syst.* (2021) 1–21, <http://dx.doi.org/10.1109/TNNLS.2021.3070843>.
- [22] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, P.S. Yu, A comprehensive survey on graph neural networks, *IEEE Trans. Neural Netw. Learn. Syst.* (2020) 1–21, <http://dx.doi.org/10.1109/TNNLS.2020.2978386>.
- [23] K. Xu, W. Hu, J. Leskovec, S. Jegelka, How powerful are graph neural networks? in: *Proc. 7th Int. Conf. Learn. Representations*, 2019.
- [24] K. Xu, J. Li, M. Zhang, S.S. Du, K. Kawarabayashi, S. Jegelka, What can neural networks reason about? in: *Proc. 8th Int. Conf. Learn. Representations*, 2020.
- [25] M. Defferrard, X. Bresson, P. Vandergheynst, Convolutional neural networks on graphs with fast localized spectral filtering, in: *Adv. Neural Inf. Process. Syst.*, 2016, pp. 3837–3845.
- [26] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *Proc. 5th Int. Conf. Learn. Representations*, 2017.
- [27] W. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Adv. Neural Inf. Process. Syst.*, 2017, pp. 1024–1034.
- [28] C. Shi, Y. Li, J. Zhang, Y. Sun, P.S. Yu, A survey of heterogeneous information network analysis, *IEEE Trans. Knowl. Data Eng.* 29 (1) (2017) 17–37.
- [29] Y. Lin, Z. Liu, X. Zhu, X. Zhu, X. Zhu, Learning entity and relation embeddings for knowledge graph completion, in: *Proc. 29th AAAI Conf. Artif. Intell.*, 2015, pp. 2181–2187.
- [30] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: *Proc. 6th Int. Conf. Learn. Representations*, 2018.
- [31] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P.S. Yu, Heterogeneous graph attention network, in: *Proc. 2019 World Wide Web Conf.*, 2019, pp. 2022–2032.
- [32] Y. Lin, Z. Liu, H. Luan, M. Sun, S. Rao, S. Liu, Modeling relation paths for representation learning of knowledge bases, in: *Proc. Conf. Empirical Methods Natural Language Process*, 2015, pp. 705–714.
- [33] J. Chen, T. Ma, C. Xiao, FastGCN: Fast learning with graph convolutional networks via importance sampling, in: *Proc. 2nd Int. Conf. Learn. Representations*, 2018.
- [34] F. Wu, J. Souza, T. Zhang, C. Fifty, T. Yu, K.Q. Weinberger, Simplifying graph convolutional networks, in: *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 6861–6871.
- [35] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: *Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [36] D. Bahdanau, K. Cho, Y. Bengio, Neural machine translation by jointly learning to align and translate, in: *Proc. 3rd Int. Conf. Learn. Representations*, 2015.
- [37] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, O. Yakhnenko, Translating embeddings for modeling multi-relational data, in: *Adv. Neural Inf. Process. Syst.*, 2013, pp. 2787–2795.
- [38] Z. Wang, J. Zhang, J. Feng, Z. Chen, Knowledge graph embedding by translating on hyperplanes, in: *Proc. 28th AAAI Conf. Artif. Intell.*, 2014, pp. 1112–1119.
- [39] J. Zheng, Q. Li, J. Liao, S. Wang, Explainable link prediction based on multi-granularity relation-embedded representation, *Knowl.-Based Syst.* 230 (2021) 107402.
- [40] G. Ji, S. He, L. Xu, K. Liu, J. Zhao, Knowledge graph embedding via dynamic mapping matrix, in: *Proc. 53rd Annu. Meeting Assoc. Comput. Linguistics 7th Int. Joint Conf. Natural Language Processing*, 2015, pp. 687–696.
- [41] T. Eblis, R. Ichise, Generalized translation-based embedding of knowledge graph, *IEEE Trans. Knowl. Data Eng.* 32 (5) (2020) 941–951.
- [42] Z. Sun, Z.-H. Deng, J.-Y. Nie, J. Tang, Rotate: Knowledge graph embedding by relational rotation in complex space, in: *Proc. 7th Int. Conf. Learn. Representations*, 2019.
- [43] X. Huang, J. Tang, Z. Tan, W. Zeng, J. Wang, X. Zhao, Knowledge graph embedding by relational and entity rotation, *Knowl. Based Syst.* 229 (2021) 107310.
- [44] B. Yang, W.-t. Yih, X. He, J. Gao, L. Deng, Embedding entities and relations for learning and inference in knowledge bases, in: *Proc. 3rd Int. Conf. Learn. Representations*, 2014.
- [45] T. Trouillon, J. Welbl, S. Riedel, É. Gaussier, G. Bouchard, Complex embeddings for simple link prediction, in: *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 2071–2080.
- [46] Y. Shen, D. Wen, Y. Li, N. Du, H. Zheng, M. Yang, Path-based attribute-aware representation learning for relation prediction, in: *Proc. 2019 SIAM Int. Conf. Data Mining*, 2019, pp. 639–647.
- [47] T. Dettmers, P. Minervini, P. Stenetorp, S. Riedel, Convolutional 2D knowledge graph embeddings, in: *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 1811–1818.
- [48] C. Shang, Y. Tang, J. Huang, J. Bi, X. He, B. Zhou, End-to-end structure-aware convolutional networks for knowledge base completion, in: *Proc. 33rd AAAI Conf. Artif. Intell.*, 2019, pp. 3060–3067.
- [49] D.Q. Nguyen, T.D. Nguyen, D.Q. Nguyen, D. Phung, A novel embedding model for knowledge base completion based on convolutional neural network, in: *Proc. Conf. North Amer. Assoc. Comput. Linguistics: Human Language Technol.*, 2018, pp. 327–333.
- [50] M. Schlichtkrull, T.N. Kipf, P. Bloem, R. Van Den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: *Proc. 15th Extended Semantic Web Conf.*, 2018, pp. 593–607.
- [51] D. Nathani, J. Chauhan, C. Sharma, M. Kaul, Learning attention-based embeddings for relation prediction in knowledge graphs, in: *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 4710–4723.
- [52] S. Vashishth, S. Sanyal, V. Nitin, P.P. Talukdar, Composition-based multi-relational graph convolutional networks, in: *Proc. 8th Int. Conf. Learn. Representations*, 2020.
- [53] A. Borrego, D. Ayala, I. Hernández, C.R. Rivero, D. Ruiz, CAFE: knowledge graph completion using neighborhood-aware features, *Eng. Appl. Artif. Intell.* 103 (2021) 104302.
- [54] J. Hu, L. Shen, S. Albanie, G. Sun, E. Wu, Squeeze-and-excitation networks, *IEEE Trans. Pattern Anal. Mach. Intell.* 42 (8) (2020) 2011–2023.
- [55] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, N. Salakhutdinov, Dropout: a simple way to prevent neural networks from overfitting, *J. Mach. Learn. Res.* 15 (1) (2014) 1929–1958.
- [56] S. Ioffe, C. Szegedy, Batch normalization: Accelerating deep network training by reducing internal covariate shift, in: *Proc. 32nd Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
- [57] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna, Rethinking the inception architecture for computer vision, in: *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, 2016, pp. 2818–2826.
- [58] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: *Proc. 3rd Int. Conf. Learn. Representations*, 2015.
- [59] K. Toutanova, D. Chen, Observed versus latent features for knowledge base and text inference, in: *Proc. 3rd Workshop Contin. Vector Space Models their Compositionality*, 2015, pp. 57–66.
- [60] W. Xiong, T. Hoang, W.Y. Wang, DeepPath: A reinforcement learning method for knowledge graph reasoning, in: *Proc. Conf. Empirical Methods Natural Language Process*, 2017, pp. 564–573.
- [61] P. Ristoski, H. Paulheim, RDF2Vec: RDF graph embeddings for data mining, in: *Proc. 15th Int. Semantic Web Conf.*, 2016, pp. 498–514.
- [62] P. Yanardag, S.V.N. Vishwanathan, Deep graph kernels, in: *Proc. 21th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2015, pp. 1365–1374.
- [63] S. Vashishth, S. Sanyal, V. Nitin, N. Agrawal, P.P. Talukdar, Interact: Improving convolution-based knowledge graph embeddings by increasing feature interactions, in: *Proc. 34th AAAI Conf. Artif. Intell.*, 2020, pp. 3009–3016.
- [64] M. Niepert, M. Ahmed, K. Kutzkov, Learning convolutional neural networks for graphs, in: M. Balcan, K.Q. Weinberger (Eds.), *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 2014–2023.

- [65] H. Paulheim, J. Fürnkranz, Unsupervised generation of data mining features from linked open data, in: *Proc. 2nd Int. Conf. Web Intell. Mining Semantics*, 2012, pp. 1–12.
- [66] M. Zhang, Z. Cui, M. Neumann, Y. Chen, An end-to-end deep learning architecture for graph classification, in: *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 4438–4445.
- [67] N. Shervashidze, P. Schweitzer, E.J. van Leeuwen, K. Mehlhorn, K.M. Borgwardt, Weisfeiler-Lehman graph Kernels, *J. Mach. Learn. Res.* 12 (2011) 2539–2561.
- [68] S. Vashishth, M. Bhandari, P. Yadav, P. Rai, C. Bhattacharyya, P.P. Talukdar, Incorporating syntactic and semantic information in word embeddings using graph convolutional networks, in: *Proc. 57th Conf. Assoc. Comput. Linguistics*, 2019, pp. 3308–3318.
- [69] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, A. Lerer, Automatic differentiation in pytorch, in: *Adv. Neural Inf. Process. Syst.*, 2017.